Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Deadlock (Kilitlenme) Kavramı
  - Karşılıklı kilitlenme: Process'lerin birbirlerini beklemesi sonucu ilerlemenin durması.
  - Process kaynakları ayrılmış durumdadır ancak instruction'ları çalıştırılamıyor. "Ölü" pozisyonda.
- Kaynak Yönetimi Modeli
  - İşletim sisteminin yönettiği her şey bir kaynaktır: İşlemci, işlemci cycle'ları, bellek alanı, G/Ç cihazları.
  - Bir kaynaktan N kopya olabilir (2 çekirdekli işlemci, 4 disk sürücüsü vb.).
  - Kaynak yönetimi: Request (talep), Use (kullan), Release (serbest bırak) prensibi.
  - Tüm kaynaklar OS üzerinden tahsis edilir; process'ler bypass edemez.
- Deadlock İçin Dört Gerekli Koşul
  - Dört koşulun hepsi aynı anda sağlanmalıdır (AND/VE ilişkisi).
  1. **Mutual Exclusion (Karşılıklı Dışlama)**: Kaynak aynı anda sadece bir process tarafından kullanılabilir.
  2. **Hold and Wait (Tut ve Bekle)**: Process bir kaynağı tutarken başka kaynaklar için bekliyor.
  3. **No Preemption (Zorla Alma Yok)**: Kaynak, process kendisi serbest bırakmadıkça zorla alınamaz.
  4. **Circular Wait (Döngüsel Bekleme)**: P0 → P1 → P2 → … → P0 şeklinde bir bekleme döngüsü.
- Resource Allocation Graph (Kaynak Atama Grafiği)
  - **Vertexler**: Process'ler (daire) ve Kaynaklar (kare, içinde nokta = kopya sayısı).
  - **Kenarlar (yönlü)**:
    - Request edge: Process → Kaynak (talep).
    - Assignment edge: Kaynak → Process (atanmış).
  - **Analiz**:
    - Kapalı çevrim YOKSA → Deadlock YOK.
    - Kapalı çevrim VARSA ve her kaynak tek kopya ise → Deadlock VAR.
    - Kapalı çevrim VARSA ve birden fazla kopya varsa → Deadlock OLABİLİR (kesin değil).
  - Örnek: P1, R2'yi tutuyor + R1'i istiyor; P2, R1'i tutuyor + R3'ü istiyor; P3, R3'ü tutuyor + R2'yi istiyor → Döngüsel bekleme → Deadlock.
- Deadlock ile Başa Çıkma Yöntemleri
  - **Deadlock Prevention (Önleme)**: Dört koşuldan en az birini ortadan kaldır.
    - Mutual exclusion'ı kaldırmak: Mümkün değil (paylaşılamaz kaynaklar var).
    - Hold and wait'i kaldırmak: Tüm kaynakları önceden talep et (low resource utilization, starvation).
    - No preemption'ı kaldırmak: Kaynağı zorla al (sadece işlemci için mümkün, dosya için değil).
    - Circular wait'i kaldırmak: Kaynakları numaralandır, artan sırayla talep et.
  - **Deadlock Avoidance (Kaçınma)**: Sistemin safe state'te kalmasını sağla.
    - Her process başlamadan önce maksimum ihtiyacını belirtir.
    - Banker algoritması: Safe sequence varsa kaynak tahsis et.
    - Safe state: Tüm process'lerin bir sırayla (P1, P2, …, Pn) ihtiyaçlarını karşılayabileceği durum.

    - Unsafe state: Deadlock olabilir ama olmak zorunda değil.
  - **Deadlock Detection (Tespit)**: Deadlock oluşmasına izin ver, sonra tespit et.
    - Resource allocation graph + wait-for graph.
    - Maliyetli: Algoritma karmaşıklığı yüksek.
  - **Deadlock Recovery (Kurtarma)**: Deadlock tespit edildikten sonra çöz.
    - Process termination: Deadlock'a dahil process'leri sonlandır.
    - Resource preemption: Kaynakları zorla al, başka process'e ver.
  - **Ostrich Algorithm (Deve Kuşu Algoritması)**: Deadlock'u görmezden gel.
    - Modern OS'lerde en yaygın yaklaşım.
    - Sebep: Algoritma maliyeti yüksek, deadlock nadir oluşuyor.

Hocanın Özellikle Vurguladığı Kısımlar

- Dört Koşulun Birlikte Sağlanması
  - Hoca vurgular: "Bu dört şartın dördü de aynı anda geçerli olmalı. Aralarında or veya ilişkisi yok. Aralarında and ve ilişkisi var. Yani bu şartlardan herhangi bir tanesi geçerli değil ise herhangi bir anda o zaman deadlock oluşamaz. Deadlock'ın oluşabilmesi için bu dört şartın dördünün de aynı anda geçerli olması lazım."
- Resource Allocation Graph Analizi
  - "Kapalı çevrim olmaması, deadlock olmaması anlamına geliyor. Güzel bir şey. Şimdi arada gidip geleceğiz dedim ya, hemen bakın bir tane ok ekledik. Yaptığımız şey çok masum gözüküyor. P3 R2'yi kullanmak istiyor. Şimdi bakın, önce kapalı çevrim hesabı yapmadan, onun takibini yapmadan genel olarak bir düşünelim."
- Kapalı Çevrim = Deadlock İhtimali
  - "Kapalı çevrim olması demek, deadlock ihtimali çok yüksek demek. Ancak her kapalı çevrim mutlaka ve mutlaka deadlock anlamına gelmiyor. Bunu da unutmamak lazım."
- Tasarım Hataları Deadlock'a Yol Açar
  - Hoca vurgular: "Deadlock neden oluşur dedik arkadaşlar? Az önce söyledik. Yanlış kullanımdan, yanlış senaryolardan, yanlış itlendirmelerden. Yani aslında tasarımı yapan, programı geliştirenlerin bir şeylere atlamasında."
- Ostrich (Deve Kuşu) Yaklaşımının Popülerliği
  - "İşletim sistemi deadlock engellemek, deadlock'tan kaçınmak veya deadlock çözmek için eğer kabul edilen sınırların, edilebilecek sınırların dışında işlem gücü harcıyorsa memoriye ihtiyaç duyuyorsa o zaman bunun bir anlamı kalmayacak. Geldiğimiz nokta bu."
- Aslında İşletim Sisteminin Garantisi
  - Hoca vurgular: "İşletim sistemi üreticileri, yazarları diyorlar ki bir işletim sisteminin içerisinde deadlock olmadığını garanti ediyoruz. Ona göre kodladık. Ona göre test ettik. Hatalarını ayıkladık. Yani kernel'ın içerisinde deadlock yok. Ama işletim sisteminin sunduğu servisleri yanlış kullanırsanız, programlama mantığınız yanlışsa, itlendirmeniz hatalıysa, konküransi seviyesi arttığında gerekli tedbirlere almadıysanız, o zaman kendi başınasınız."
- Temel Bileşen Sayısı Önemli
  - "En basitinden Cihan Hoca'nız geçen haftalarda da söylemişti, bir Apache Web Server'ı düşündüğünüzde Apache Web Server başladığında kendinden on tane proses oluşturuyor, o proseslerin içinde de diyelim ki beşer tane thread oluyor, oluyor size bir anda elli tane kopya. Bunların birbirleriyle etkileşmemesi mümkün değil."

Kısa Tekrar Notları

- Deadlock: karşılıklı kilitlenme, tüm süreçler bloke.
- 4 deadlock koşulu: mutual exclusion, hold and wait, no preemption, circular wait.
- Resource allocation graph: process (daire) + resource (kare).
- Kapalı çevrim → deadlock ihtimali.
- Prevention: koşullardan birini kaldır.
- Avoidance: safe state'te tut (Banker).
- Detection: deadlock oluştuktan sonra tespit.
- Recovery: process termination veya resource preemption.
- Ostrich: deadlock'u görmezden gel (modern OS'lerde yaygın).

Detaylı Açıklamalar

Ders 12, deadlock (kilitlenme) kavramını derinlemesine ele alır. Bu ders, senkronizasyon konularının doğal bir uzantısıdır: process'ler senkronizasyon hataları yüzünden sonsuza kadar birbirlerini bekleyebilir.

Deadlock, process'lerin birbirlerini beklemesi sonucu ilerlemenin durmasıdır. Process kaynakları ayrılmış durumdadır ancak instruction'ları çalıştırılamıyor. Bu "ölü" pozisyondan çıkmak için dış müdahale gerekir.

Deadlock'un oluşabilmesi için dört koşulun hepsinin aynı anda sağlanması gerekir:

1. **Mutual Exclusion (Karşılıklı Dışlama)**: Kaynak aynı anda sadece bir process tarafından kullanılabilir. Bazı kaynaklar doğası gereği paylaşılamaz (yazıcı, dosya).
2. **Hold and Wait (Tut ve Bekle)**: Process bir kaynağı tutarken başka kaynaklar için bekliyor. Örnek: process disk'i tutuyor, yazıcı bekliyor.
3. **No Preemption (Zorla Alma Yok)**: Kaynak, process kendisi serbest bırakmadıkça zorla alınamaz. İşlemci dışında bu genellikle mümkün değil.
4. **Circular Wait (Döngüsel Bekleme)**: P0 → P1 → P2 → ... → P0 şeklinde bir bekleme döngüsü. Her process bir sonrakini bekliyor.

Resource Allocation Graph, deadlock analizi için kullanılan grafiksel araçtır. Process'ler daire, kaynaklar kare ile gösterilir (kare içindeki noktalar kopya sayısını gösterir). Request edge (process → kaynak) talebi, assignment edge (kaynak → process) atamayı gösterir. Kapalı çevrim yoksa deadlock yoktur. Kapalı çevrim varsa ve her kaynak tek kopya ise deadlock vardır. Kapalı çevrim olup birden fazla kopya varsa deadlock olabilir ama kesin değildir.

Deadlock ile başa çıkma yöntemleri dört kategoride incelenir:

Prevention (Önleme): Dört koşulun en az birini ortadan kaldırmak. Ancak her koşul kaldırılamaz: mutual exclusion'ı kaldırmak çoğu kaynak için mümkün değildir; hold and wait'i kaldırmak tüm kaynakları önceden talep etmeyi gerektirir (kaynak israfı); no preemption'ı kaldırmak sadece bazı kaynaklar için mümkündür; circular wait'i kaldırmak en pratik yöntemdir (kaynakları numaralandırma).

Avoidance (Kaçınma): Sistemin safe state'te kalmasını sağlamak. Her process başlamadan önce maksimum ihtiyacını belirtir. Banker algoritması safe sequence arar. Safe state: tüm process'lerin bir sırayla ihtiyaçlarını karşılayabileceği durum. Unsafe state: deadlock olabilir ama zorunda değil. Safe → kesin deadlock yok; Unsafe → deadlock olasılığı.

Detection (Tespit): Deadlock oluşmasına izin ver, sonra tespit et. Resource allocation graph + wait-for graph kullanılır. Algoritma karmaşıklığı yüksektir.

Recovery (Kurtarma): Deadlock tespit edildikten sonra çözmek. Process termination: deadlock'a dahil process'leri sonlandır. Resource preemption: kaynakları zorla al.

Ostrich Algorithm (Deve Kuşu): Deadlock'u görmezden gelmek. Modern OS'lerde en yaygın yaklaşımdır çünkü deadlock nadir oluşur, algoritma maliyeti yüksektir ve kernel seviyesinde deadlock olmadığı garanti edilir. Sorun ancak uygulama seviyesinde yanlış kullanımdan doğar.

Hoca, modern OS'lerin bu konudaki yaklaşımını açıklar: "İşletim sistemi deadlock engellemek, deadlock'tan kaçınmak veya deadlock çözmek için eğer kabul edilen sınırların dışında işlem gücü harcıyorsa, o zaman bunun bir anlamı kalmayacak. Geldiğimiz nokta bu."

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.